# Chapter 5. Transaction Processing and Recovery

In this book, we've taken a bottom-up approach to database system concepts: we first learned about storage structures. Now, we're ready to move to the higher-level components responsible for buffer management, lock management, and recovery, which are the prerequisites for understanding database transactions.

A *transaction* is an indivisible logical unit of work in a database management system, allowing you to represent multiple operations as a single step. Operations executed by transactions include reading and writing database records. A database transaction has to preserve atomicity, consistency, isolation, and durability. These properties are commonly referred as *ACID* [HAERDER83]:

*Atomicity*

Transaction steps are *indivisible*, which means that either *all* the steps associated with the transaction execute successfully or none of them do. In other words, transactions should not be applied partially. Each transaction can either *commit* (make all changes from write operations executed during the transaction visible), or *abort* (roll back all transaction side effects that haven't yet been made visible). Commit is a final operation. After an abort, the transaction can be retried.

*Consistency*

Consistency is an application-specific guarantee; a transaction should only bring the database from one valid state to another valid state, maintaining all database invariants (such as constraints, referential integrity, and others). Consistency is the most weakly defined property, possibly because it is the only property that is controlled by the user and not only by the database itself.

*Isolation*

Multiple concurrently executing transactions should be able to run

without interference, as if there were no other transactions executing at the same time. Isolation defines *when* the changes to the database state may become visible, and what changes may become visible to the concurrent transactions. Many databases use isolation levels that are weaker than the given definition of isolation for performance reasons. Depending on the methods and approaches used for concurrency control, changes made by a transaction may or may not be visible to other concurrent transactions (see "Isolation Levels").

*Durability*

Once a transaction has been committed, all database state modifications have to be persisted on disk and be able to survive power outages, system failures, and crashes.

Implementing transactions in a database system, in addition to a storage structure that organizes and persists data on disk, requires several components to work together. On the node locally, the *transaction manager* coordinates, schedules, and tracks transactions and their individual steps.

The *lock manager* guards access to these resources and prevents concurrent accesses that would violate data integrity. Whenever a lock is requested, the lock manager checks if it is already held by any other transaction in shared or exclusive mode, and grants access to it if the requested access level results in no contradiction. Since exclusive locks can be held by at most one transaction at any given moment, other transactions requesting them have to wait until locks are released, or abort and retry later. As soon as the lock is released or whenever the transaction terminates, the lock manager notifies one of the pending transactions, letting it acquire the lock and continue.

The *page cache* serves as an intermediary between persistent storage (disk) and the rest of the storage engine. It stages state changes in main memory and serves as a cache for the pages that haven't been synchronized with persistent storage. All changes to a database state are first applied to the cached pages.

The *log manager* holds a history of operations (log entries) applied to cached pages but not yet synchronized with persistent storage to guarantee they won't be lost in case of a crash. In other words, the log is used to

reapply these operations and reconstruct the cached state during startup. Log entries can also be used to undo changes done by the aborted transactions.

Distributed (multipartition) transactions require additional coordination and remote execution. We discuss distributed transaction protocols in Chapter 13.

## Buffer Management

Most databases are built using a two-level memory hierarchy: slower persistent storage (disk) and faster main memory (RAM). To reduce the number of accesses to persistent storage, pages are *cached* in memory. When the page is requested again by the storage layer, its cached copy is returned.

Cached pages available in memory can be reused under the assumption that no other process has modified the data on disk. This approach is sometimes referenced as *virtual disk* [BAYER72]. A virtual disk read accesses physical storage only if no copy of the page is already available in memory. A more common name for the same concept is *page cache* or *buffer pool*. The page cache is responsible for caching pages read from disk in memory. In case of a database system crash or unorderly shutdown, cached contents are lost.

Since the term *page cache* better reflects the purpose of this structure, this book defaults to this name. The term *buffer pool* sounds like its primary purpose is to pool and reuse *empty* buffers, without sharing their contents, which can be a useful part of a page cache or even as a separate component, but does not reflect the entire purpose as precisely.

The problem of caching pages is not limited in scope to databases. Operating systems have the concept of a page cache, too. Operating systems utilize *unused* memory segments to transparently cache disk contents to improve performance of I/O syscalls.

Uncached pages are said to be *paged in* when they're loaded from disk. If any changes are made to the cached page, it is said to be *dirty*, until these changes are *flushed* back on disk.

Since the memory region where cached pages are held is usually substantially smaller than an entire dataset, the page cache eventually fills up and, in order to page in a new page, one of the cached pages has to be *evicted*.

In Figure 5-1, you can see the relation between the logical representation of B-Tree pages, their cached versions, and the pages on disk. The page cache loads pages into free slots out of order, so there's no direct mapping

between how pages are ordered on disk and in memory.

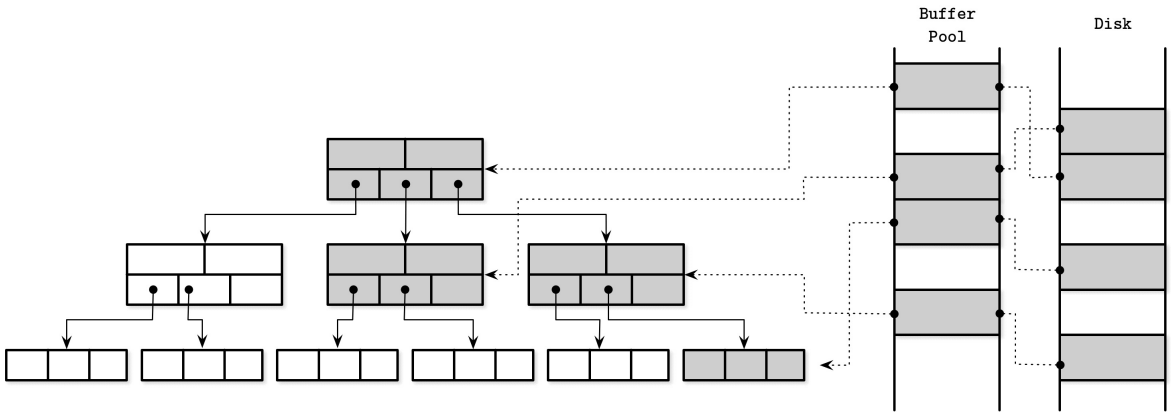


*Figure 5-1. Page cache*

The primary functions of a page cache can be summarized as:

- It keeps cached page contents in memory.
- It allows modifications to on-disk pages to be *buffered* together and performed against their cached versions.
- When a requested page isn't present in memory and there's enough space available for it, it is *paged in* by the page cache, and its cached version is returned.
- If an already cached page is requested, its cached version is returned.
- If there's not enough space available for the new page, some other page is *evicted* and its contents are *flushed* to disk.

**BYPASSING THE KERNEL PAGE CACHE**

Many database systems open files using `O_DIRECT` flag. This flag allows I/O system calls to bypass the kernel page cache, access the disk directly, and use database-specific buffer management. This is sometimes frowned upon by the operating systems folks.

Linus Torvalds has criticized usage of `O_DIRECT` since it's not asynchronous and has no readahead or other means for instructing the kernel about access patterns. However, until operating systems start offering better mechanisms, `O_DIRECT` is still going to be useful.

We can gain some control over how the kernel evicts pages from its cache is by using `fadvise`, but this only allows us to ask the kernel to consider our opinion and does not guarantee it will actually happen. To avoid syscalls when performing I/O, we can use memory mapping, but then we lose control over caching.

## Caching Semantics

All changes made to buffers are kept in memory until they are eventually written back to disk. As no other process is allowed to make changes to the backing file, this synchronization is a one-way process: from memory to disk, and not vice versa. The page cache allows the database to have more control over memory management and disk accesses. You can think of it as an application-specific equivalent of the kernel page cache: it accesses the block device directly, implements similar functionality, and serves a similar purpose. It abstracts disk accesses and decouples logical write operations from the physical ones.

Caching pages helps to keep the tree partially in memory without making additional changes to the algorithm and materializing objects in memory. All we have to do is replace disk accesses by the calls to the page cache.

When the storage engine accesses (in other words, requests) the page, we first check if its contents are already cached, in which case the cached page contents are returned. If the page contents are not yet cached, the cache translates the logical page address or page number to its physical address, loads its contents in memory, and returns its cached version to the storage engine. Once returned, the buffer with cached page contents is said to be *referenced*, and the storage engine has to hand it back to the page cache or dereference it once it’s done. The page cache can be instructed to avoid evicting pages by *pinning* them.

If the page is modified (for example, a cell was appended to it), it is marked as dirty. A dirty flag set on the page indicates that its contents are out of sync with the disk and have to be flushed for durability.

## Cache Eviction

Keeping caches populated is good: we can serve more reads without going to persistent storage, and more same-page writes can be buffered together. However, the page cache has a limited capacity and, sooner or later, to serve the new contents, old pages have to be evicted. If page contents are in sync with the disk (i.e., were already flushed or were never modified) and the page is not pinned or referenced, it can be evicted right away. Dirty pages have to be *flushed* before they can be evicted. Referenced pages should not be evicted while some other thread is using them.

Since triggering a flush on every eviction might be bad for performance, some databases use a separate background process that cycles through the dirty pages that are likely to be evicted, updating their disk versions. For example, PostgreSQL has a background flush writer that does just that.

Another important property to keep in mind is *durability*: if the database has crashed, all data that was not flushed is lost. To make sure that all changes are persisted, flushes are coordinated by the *checkpoint* process. The checkpoint process controls the write-ahead log (WAL) and page cache, and ensures that they work in lockstep. Only log records associated with operations applied to cached pages that were flushed can be discarded from the WAL. Dirty pages cannot be evicted until this process completes.

This means there is always a trade-off between several objectives:

- Postpone flushes to reduce the number of disk accesses
- Preemptively flush pages to allow quick eviction
- Pick pages for eviction and flush in the optimal order
- Keep cache size within its memory bounds
- Avoid losing the data as it is not persisted to the primary storage

We explore several techniques that help us to improve the first three characteristics while keeping us within the boundaries of the other two.

## Locking Pages in Cache

Having to perform disk I/O on each read or write is impractical: subsequent reads may request the same page, just as subsequent writes may modify the same page. Since B-Tree gets "narrower" toward the top, higher-level nodes (ones that are closer to the root) are hit for most of the reads. Splits and merges also eventually propagate to the higher-level nodes. This means there's always at least a part of a tree that can significantly benefit from being cached.

We can "lock" pages that have a high probability of being used in the nearest time. Locking pages in the cache is called *pinning*. Pinned pages are kept in memory for a longer time, which helps to reduce the number of disk accesses and improve performance [GRAEFE11].

Since each lower B-Tree node level has exponentially more nodes than the higher one, and higher-level nodes represent just a small fraction of the tree, this part of the tree can reside in memory permanently, and other parts can be paged in on demand. This means that, in order to perform a query, we won't have to make $h$ disk accesses (as discussed in "B-Tree Lookup Complexity", $h$ is the height of the tree), but only hit the disk for the lower levels, for which pages are not cached.

Operations performed against a subtree may result in structural changes that contradict each other — for example, multiple delete operations causing merges followed by writes causing splits, or vice versa. Likewise for structural changes that propagate from different subtrees (structural changes occurring close to each other in time, in different parts of the tree, propagating up). These operations can be buffered together by applying changes only in memory, which can reduce the number of disk writes and amortize the operation costs, since only one write can be performed instead of multiple writes.

**PREFETCHING AND IMMEDIATE EVICTION**

The page cache also allows the storage engine to have fine-grained control over prefetching and eviction. It can be instructed to load pages ahead of time, before they are accessed. For example, when the leaf nodes are traversed in a range scan, the next leaves can be preloaded. Similarly, if a maintenance process loads the page, it can be evicted immediately after the process finishes, since it's unlikely to be useful for the in-flight queries. Some databases, for

example, PostgreSQL, use a circular buffer (in other words, FIFO page replacement policy) for large sequential scans.

## Page Replacement

When cache capacity is reached, to load new pages, old ones have to be evicted. However, unless we evict pages that are least likely to be accessed again soon, we might end up loading them several times subsequently even though we could've just kept them in memory for all that time. We need to find a way to estimate the likelihood of subsequent page access to optimize this.

For this, we can say that pages should be evicted according to the *eviction policy* (also sometimes called the *page-replacement* policy). It attempts to find pages that are least likely to be accessed again any time soon. When the page is evicted from the cache, the new page can be loaded in its place.

For a page cache implementation to be performant, it needs an efficient page-replacement algorithm. An ideal page-replacement strategy would require a crystal ball that would predict the order in which pages are going to be accessed and evict only pages that will not be touched for the longest time. Since requests do not necessarily follow any specific pattern or distribution, precisely predicting behavior can be complicated, but using a right page replacement strategy can help to reduce the number of evictions.

It seems logical that we can reduce the number of evictions by simply using a larger cache. However, this does not appear to be the case. One of the examples demonstrating this dilemma this is called *Bélády's anomaly* [BEDALY69]. It shows that increasing the number of pages might increase the number of evictions if the used page-replacement algorithm is not optimal. When pages that might be required soon are evicted and then loaded again, pages start competing for space in the cache. Because of that, we need to wisely consider the algorithm we're using, so that it would improve the situation, not make it worse.

### FIFO and LRU

The most naïve page-replacement strategy is first in, first out (*FIFO*). FIFO maintains a queue of page IDs in their insertion order, adding new pages to the tail of the queue. Whenever the page cache is full, it takes the element from the head of the queue to find the page that was paged in at the farthest point in time. Since it does not account for subsequent page

accesses, only for page-in events, this proves to be impractical for the most real-world systems. For example, the root and topmost-level pages are paged in first and, according to this algorithm, are the first candidates for eviction, even though it's clear from the tree structure that these pages are likely to paged in again soon, if not immediately.

A natural extension of the FIFO algorithm is *least-recently used* (LRU) [TANENBAUM14]. It also maintains a queue of eviction candidates in insertion order, but allows you to place a page back to the tail of the queue on repeated accesses, as if this was the first time it was paged in. However, updating references and relinking nodes on every access can become expensive in a concurrent environment.

There are other LRU-based cache eviction strategies. For example, 2Q (Two-Queue LRU) maintains two queues and puts pages into the first queue during the initial access and moves them to the second *hot* queue on subsequent accesses, allowing you to distinguish between the recently and frequently accessed pages [JONSON94]. LRU-K identifies frequently referenced pages by keeping track of the last `K` accesses, and using this information to estimate access times on a page basis [ONEIL93].

### CLOCK

In some situations, efficiency may be more important than precision. *CLOCK* algorithm variants are often used as compact, cache-friendly, and concurrent alternatives to LRU [SOUNDARARARJAN06]. Linux, for example, uses a variant of the CLOCK algorithm.

CLOCK-sweep holds references to pages and associated access bits in a circular buffer. Some variants use counters instead of bits to account for frequency. Every time the page is accessed, its access bit is set to `1`. The algorithm works by going around the circular buffer, checking access bits:

- If the access bit is `1`, and the page is unreferenced, it is set to `0`, and the next page is inspected.
- If the access bit is already `0`, the page becomes a *candidate* and is scheduled for eviction.
- If the page is currently referenced, its access bit remains unchanged. It is assumed that the access bit of an accessed page cannot be 0, so it

cannot be evicted. This makes referenced pages less likely to be replaced.

Figure 5-2 shows a circular buffer with access bits.

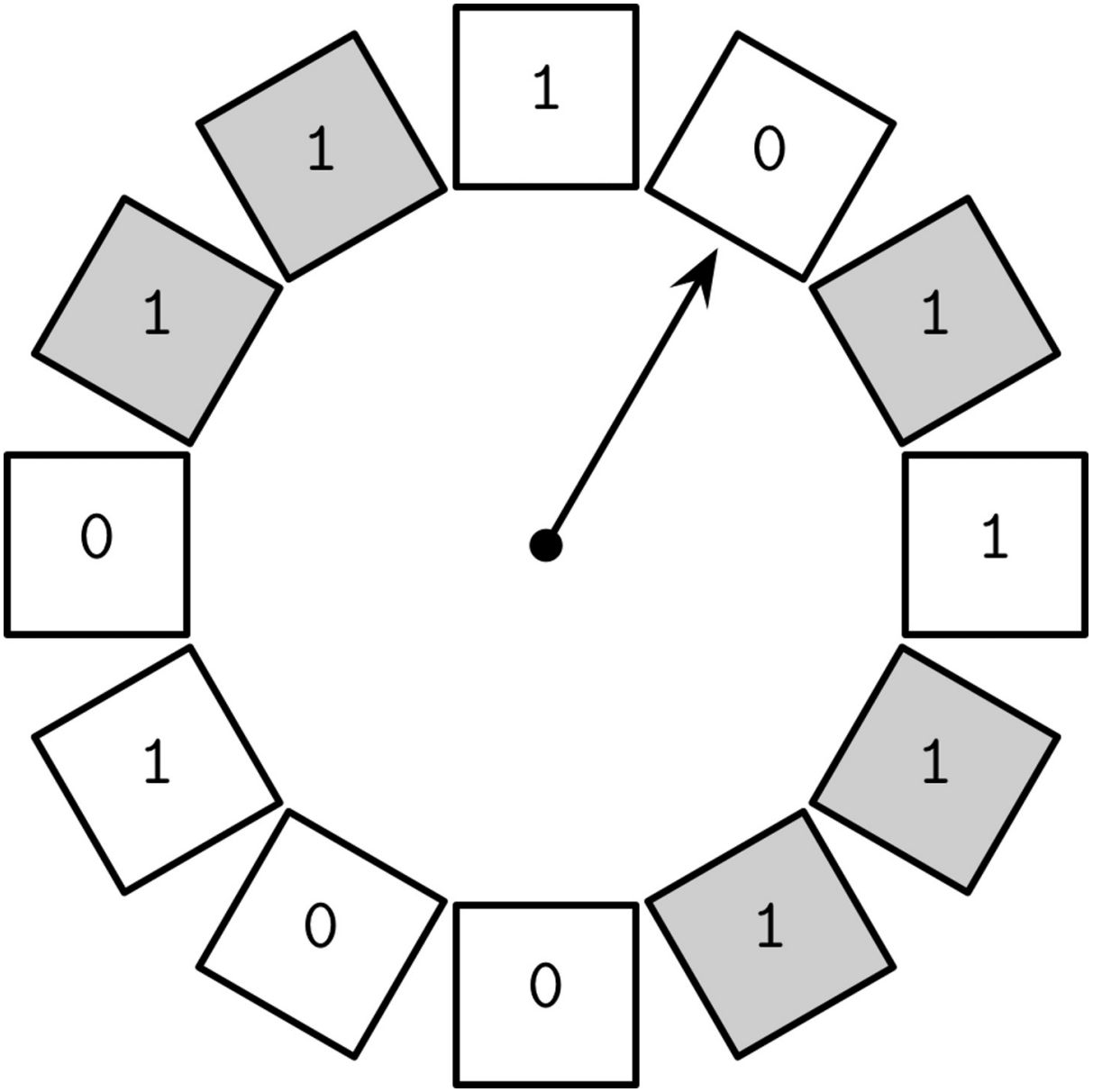


*Figure 5-2. CLOCK-sweep example. Counters for currently referenced pages are shown in gray. Counters for unreferenced pages are shown in white. The arrow points to the element that will be checked next.*

An advantage of using a circular buffer is that both the clock hand pointer and contents can be modified using compare-and-swap operations, and do not require additional locking mechanisms. The algorithm is easy to understand and implement and is often used in both textbooks [TANENBAUM14] and real-wold systems.

LRU is not always the best replacement strategy for a database system. Sometimes, it may be more practical to consider *usage frequency* rather

than *recency* as a predictive factor. In the end, for a database system under a heavy load, recency might not be very indicative as it only represents the order in which items were accessed.

### LFU

To improve the situation, we can start tracking *page reference events* rather than *page-in events*. One of the approaches allowing us to do this tracks least-frequently used (LFU) pages.

TinyLFU, a frequency-based page-eviction policy [EINZIGER17], does precisely this: instead of evicting pages based on *page-in recency*, it orders pages by *usage frequency*. It is implemented in the popular Java library called Caffeine.

TinyLFU uses a frequency histogram [CORMODE11] to maintain compact cache access history, since preserving an entire history might be prohibitively expensive for practical purposes.

Elements can be in one of the three queues:

- *Admission*, maintaining newly added elements, implemented using LRU policy.
- *Probation*, holding elements most likely to get evicted.
- *Protected*, holding elements that are to stay in the queue for a longer time.

Rather than choosing which elements to evict every time, this approach chooses which ones to promote for retention. Only the items that have a frequency larger than the item that would be evicted as a result of promoting them, can be moved to the probation queue. On subsequent accesses, items can get moved from probation to the protected queue. If the protected queue is full, one of the elements from it may have to be placed back into probation. More frequently accessed items have a higher chance of retention, and less frequently used ones are more likely to be evicted.

Figure 5-3 shows the logical connections between the admission, probation, and protected queues, the frequency filter, and eviction.

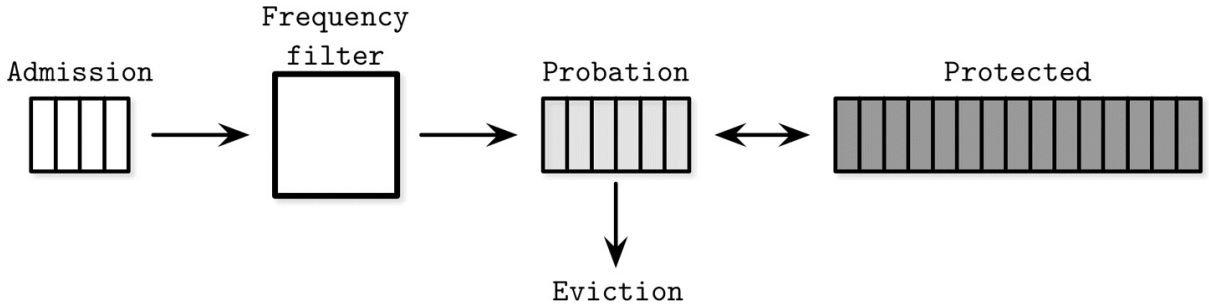


*Figure 5-3. TinyLFU admission, protected, and probation queues*

There are many other algorithms that can be used for optimal cache eviction. The choice of a page-replacement strategy has a significant impact on latency and the number of performed I/O operations, and has to be taken into consideration.

# Recovery

Database systems are built on top of several hardware and software layers that can have their own stability and reliability problems. Database systems themselves, as well as the underlying software and hardware components, may fail. Database implementers have to consider these failure scenarios and make sure that the data that was "promised" to be written is, in fact, written.

A *write-ahead log* (WAL for short, also known as a *commit log*) is an append-only auxiliary disk-resident structure used for crash and transaction recovery. The page cache allows buffering changes to page contents in memory. Until the cached contents are flushed back to disk, the only disk-resident copy preserving the operation history is stored in the WAL. Many database systems use append-only write-ahead logs; for example, PostgreSQL and MySQL.

The main functionality of a write-ahead log can be summarized as:

- Allow the page cache to buffer updates to disk-resident pages while ensuring durability semantics in the larger context of a database system.

- Persist all operations on disk until the cached copies of pages affected by these operations are synchronized on disk. Every operation that modifies the database state has to be logged on disk *before* the contents of the associated pages can be modified.

- Allow lost in-memory changes to be reconstructed from the operation log in case of a crash.

In addition to this functionality, the write-ahead log plays an important role in transaction processing. It is hard to overstate the importance of the WAL as it ensures that data makes it to the persistent storage and is available in case of a crash, as uncommitted data is replayed from the log and the pre-crash database state is fully restored. In this section, we will often refer to ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), a state-of-the-art recovery algorithm that is widely used and cited [MOHAN92].

### POSTGRESQL VERSUS FSYNC()

PostgreSQL uses checkpoints to ensure that index and data files have been updated with all information up to a certain record in the logfile. Flushing all dirty (modified) pages at once is done periodically by the checkpoint process. Synchronizing dirty page contents with disk is done by making the `fsync()` kernel call, which is supposed to sync dirty pages to disk, and unset the *dirty* flag on the kernel pages. As you would expect, `fsync` returns with an error if it isn't able to flush pages on disk.

In Linux and a few other operating systems, `fsync` unsets the dirty flag *even* from unsuccessfully flushed pages after I/O errors. Additionally, errors will be reported only to the file descriptors that were open at the time of failure, so `fsync` will *not* return any errors that have occurred *before* the descriptor it was called upon was opened [CORBET18].

Since the checkpointer doesn't keep all files open at any given point in time, it may happen that it misses error notifications. Because dirty page flags are cleared, the checkpointer will assume that data has successfully made it on disk while, in fact, it might have not been written.

A combination of these behaviors can be a source of data loss or database corruption in the presence of potentially recoverable failures. Such behaviors can be difficult to detect and some of the states they lead to may be unrecoverable. Sometimes, even triggering such behavior can be nontrivial. When working on recovery mechanisms, we should always take extra care and think through and attempt to test every possible failure scenario.

## Log Semantics

The write-ahead log is append-only and its written contents are immutable, so all writes to the log are sequential. Since the WAL is an immutable, append-only data structure, readers can safely access its contents up to the latest write threshold while the writer continues appending data to the log tail.

The WAL consists of log records. Every record has a unique, monotonically increasing *log sequence number* (LSN). Usually, the LSN is represented by an internal counter or a timestamp. Since log records do not necessarily occupy an entire disk block, their contents are cached in the *log buffer* and are flushed on disk in a *force* operation. Forces happen as the log buffers fill up, and can be requested by the transaction manager or a page cache. All log records have to be flushed on disk in LSN order.

Besides individual operation records, the WAL holds records indicating transaction completion. A transaction can't be considered committed until the log is forced up to the LSN of its commit record.

To make sure the system can continue functioning correctly after a crash during rollback or recovery, some systems use *compensation log records* (CLR) during undo and store them in the log.

The WAL is usually coupled with a primary storage structure by the interface that allows *trimming* it whenever a *checkpoint* is reached. Logging is one of the most critical correctness aspects of the database, which is somewhat tricky to get right: even the slightest disagreements between log trimming and ensuring that the data has made it to the primary storage structure may cause data loss.

Checkpoints are a way for a log to know that log records up to a certain mark are fully persisted and aren't required anymore, which significantly reduces the amount of work required during the database startup. A process that forces *all* dirty pages to be flushed on disk is generally called a *sync checkpoint*, as it fully synchronizes the primary storage structure.

Flushing the entire contents on disk is rather impractical and would require pausing all running operations until the checkpoint is done, so most database systems implement *fuzzy checkpoints*. In this case, the

`last_checkpoint` pointer stored in the log header contains the information about the last successful checkpoint. A fuzzy checkpoint begins with a special `begin_checkpoint` log record specifying its start, and ends with `end_checkpoint` log record, containing information about the dirty pages, and the contents of a transaction table. Until all the pages specified by this record are flushed, the checkpoint is considered to be *incomplete*. Pages are flushed asynchronously and, once this is done, the `last_checkpoint` record is updated with the LSN of the `begin_checkpoint` record and, in case of a crash, the recovery process will start from there [MOHAN92].

## Operation Versus Data Log

Some database systems, for example System R [CHAMBERLIN81], use *shadow paging*: a copy-on-write technique ensuring data durability and transaction atomicity. New contents are placed into the new unpublished *shadow* page and made visible with a pointer flip, from the old page to the one holding updated contents.

Any state change can be represented by a before-image and an after-image or by corresponding redo and undo operations. Applying a *redo* operation to a *before-image* produces an *after-image*. Similarly, applying an *undo* operation to an *after-image* produces a *before-image*.

We can use a physical log (that stores complete page state or byte-wise changes to it) or a logical log (that stores operations that have to be performed against the current state) to move records or pages from one state to the other, both backward and forward in time. It is important to track the *exact* state of the pages that physical and logical log records can be applied to.

Physical logging records before and after images, requiring entire pages affected by the operation to be logged. A logical log specifies which operations have to be applied to the page, such as `"insert a data record X for key Y"`, and a corresponding undo operation, such as `"remove the value associated with Y"`.

In practice, many database systems use a combination of these two approaches, using logical logging to perform an undo (for concurrency and performance) and physical logging to perform a redo (to improve recovery time) [MOHAN92].

## Steal and Force Policies

To determine when the changes made in memory have to be flushed on disk, database management systems define steal/no-steal and force/no-force policies. These policies are *mostly* applicable to the page cache, but they're better discussed in the context of recovery, since they have a significant impact on which recovery approaches can be used in combination with them.

A recovery method that allows flushing a page modified by the transaction even before the transaction has committed is called a *steal* policy. A *no-steal* policy does not allow flushing any uncommitted transaction contents on disk. To *steal* a dirty page here means flushing its in-memory contents to disk and loading a different page from disk in its place.

A *force* policy requires all pages modified by the transactions to be flushed on disk *before* the transaction commits. On the other hand, a *no-force* policy allows a transaction to commit even if some pages modified during this transaction were not yet flushed on disk. To *force* a dirty page here means to flush it on disk before the commit.

Steal and force policies are important to understand, since they have implications for transaction undo and redo. *Undo* rolls back updates to forced pages for committed transactions, while *redo* applies changes performed by committed transactions on disk.

Using the *no-steal* policy allows implementing recovery using only redo entries: old copy is contained in the page on disk and modification is stored in the log [WEIKUM01]. With *no-force*, we potentially can buffer several updates to pages by *deferring* them. Since page contents have to be cached in memory for that time, a larger page cache may be needed.

When the *force* policy is used, crash recovery doesn't need any additional work to reconstruct the results of committed transactions, since pages modified by these transactions are already flushed. A major drawback of using this approach is that transactions take longer to commit due to the necessary I/O.

More generally, *until* the transaction commits, we need to have enough information to undo its results. If any pages touched by the transaction are

flushed, we need to keep undo information in the log until it commits to be able to roll it back. Otherwise, we have to keep redo records in the log until it commits. In both cases, transaction *cannot* commit until either undo or redo records are written to the logfile.

## ARIES

ARIES is a *steal/no-force* recovery algorithm. It uses physical redo to improve performance during recovery (since changes can be installed quicker) and logical undo to improve concurrency during normal operation (since logical undo operations can be applied to pages independently). It uses WAL records to implement *repeating history* during recovery, to completely reconstruct the database state before undoing uncommitted transactions, and creates compensation log records during undo [MOHAN92].

When the database system restarts after the crash, recovery proceeds in three phases:

1. The *analysis* phase identifies dirty pages in the page cache and transactions that were in progress at the time of a crash. Information about dirty pages is used to identify the starting point for the redo phase. A list of in-progress transactions is used during the undo phase to roll back incomplete transactions.

2. The *redo* phase repeats the history up to the point of a crash and restores the database to the previous state. This phase is done for incomplete transactions as well as ones that were committed but whose contents weren't flushed to persistent storage.

3. The *undo* phase rolls back all incomplete transactions and restores the database to the last consistent state. All operations are rolled back in reverse chronological order. In case the database crashes again during recovery, operations that undo transactions are logged as well to avoid repeating them.

ARIES uses LSNs for identifying log records, tracks pages modified by running transactions in the dirty page table, and uses physical redo, logical undo, and fuzzy checkpointing. Even though the paper describing this system was released in 1992, most concepts, approaches, and paradigms are still relevant in transaction processing and recovery today.

# Concurrency Control

When discussing database management system architecture in "DBMS Architecture", we mentioned that the transaction manager and lock manager work together to handle *concurrency control*. Concurrency control is a set of techniques for handling interactions between concurrently executing transactions. These techniques can be roughly grouped into the following categories:

*Optimistic concurrency control (OCC)*

Allows transactions to execute concurrent read and write operations, and determines whether or not the result of the combined execution is serializable. In other words, transactions do not block each other, maintain histories of their operations, and check these histories for possible conflicts before commit. If execution results in a conflict, one of the conflicting transactions is aborted.

*Multiversion concurrency control (MVCC)*

Guarantees a consistent view of the database at some point in the past identified by the timestamp by allowing multiple timestamped versions of the record to be present. MVCC can be implemented using validation techniques, allowing only one of the updating or committing transactions to win, as well as with lockless techniques such as timestamp ordering, or lock-based ones, such as two-phase locking.

*Pessimistic (also known as conservative) concurrency control (PCC)*

There are both lock-based and nonlocking conservative methods, which differ in how they manage and grant access to shared resources. Lock-based approaches require transactions to maintain locks on database records to prevent other transactions from modifying locked records and assessing records that are being modified until the transaction releases its locks. Nonlocking approaches maintain read and write operation lists and restrict execution, depending on the schedule of unfinished transactions. Pessimistic schedules can result in a deadlock when multiple transactions wait for each other to release a lock in order to proceed.

In this chapter, we concentrate on node-local concurrency control techniques. In Chapter 13, you can find information about distributed transactions and other approaches, such as deterministic concurrency control (see “Distributed Transactions with Calvin”).

Before we can further discuss concurrency control, we need to define a set of problems we’re trying to solve and discuss how transaction operations overlap and what consequences this overlapping has.

## Serializability

Transactions consist of read and write operations executed against the database state, and business logic (transformations, applied to the read contents). A *schedule* is a list of operations required to execute a set of transactions from the database-system perspective (i.e., only ones that interact with the database state, such as read, write, commit, or abort operations), since all other operations are assumed to be side-effect free (in other words, have no impact on the database state) [MOLINA08].

A schedule is *complete* if contains all operations from every transaction executed in it. *Correct* schedules are logical equivalents to the original lists of operations, but their parts can be executed in parallel or get reordered for optimization purposes, as long as this does not violate ACID properties and the correctness of the results of individual transactions [WEIKUM01].

A schedule is said to be *serial* when transactions in it are executed completely independently and without any interleaving: every preceding transaction is fully executed before the next one starts. Serial execution is easy to reason about, as contrasted with all possible interleavings between several multistep transactions. However, always executing transactions one after another would significantly limit the system throughput and hurt performance.

We need to find a way to execute transaction operations concurrently, while maintaining the correctness and simplicity of a serial schedule. We can achieve this with *serializable* schedules. A schedule is serializable if it is equivalent to *some* complete serial schedule over the same set of transactions. In other words, it produces the same result as if we executed a set of transactions one after another in *some* order. Figure 5-4 shows three concurrent transactions, and possible execution histories (`3! = 6` possibilities, in every possible order).

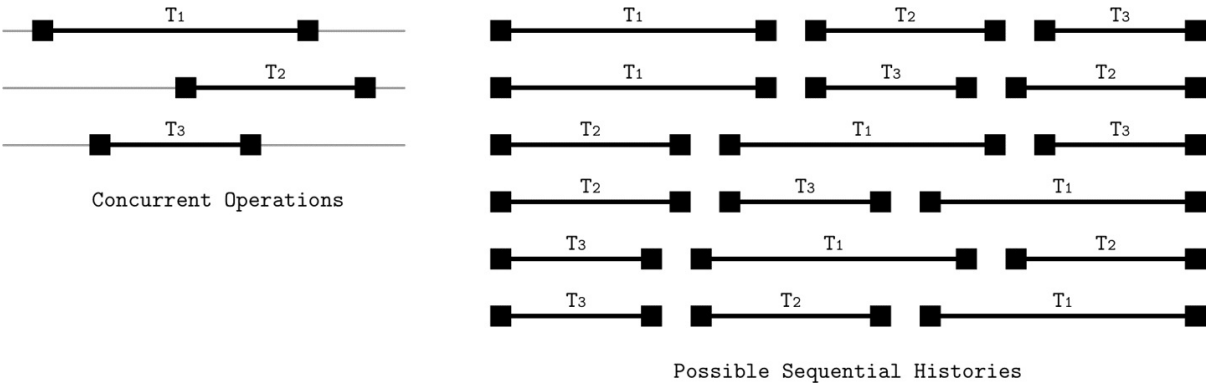


*Figure 5-4. Concurrent transactions and their possible sequential execution histories*

## Transaction Isolation

Transactional database systems allow different isolation levels. An *isolation level* specifies how and when parts of the transaction can and should become visible to other transactions. In other words, isolation levels describe the degree to which transactions are isolated from other concurrently executing transactions, and what kinds of anomalies can be encountered during execution.

Achieving isolation comes at a cost: to prevent incomplete or temporary writes from propagating over transaction boundaries, we need additional coordination and synchronization, which negatively impacts the performance.

## Read and Write Anomalies

The SQL standard [MELTON06] refers to and describes *read anomalies* that can occur during execution of concurrent transactions: dirty, nonrepeatable, and phantom reads.

A *dirty read* is a situation in which a transaction can read uncommitted changes from other transactions. For example, transaction $T_1$ updates a user record with a new value for the address field, and transaction $T_2$ reads the updated address before $T_1$ commits. Transaction $T_1$ aborts and rolls back its execution results. However, $T_2$ has already been able to read this value, so it has accessed the value that has never been committed.

A *nonrepeatable read* (sometimes called a fuzzy read) is a situation in which a transaction queries the *same row* twice and gets different results. For example, this can happen even if transaction $T_1$ reads a row, then transaction $T_2$ modifies it *and commits* this change. If $T_1$ requests the same row again before finishing its execution, the result will differ from the previous run.

If we use range reads during the transaction (i.e., read not a single data record, but a range of records), we might see *phantom records*. A *phantom read* is when a transaction queries the same *set of rows* twice and receives different results. It is similar to a nonrepeatable read, but holds for range queries.

There are also *write anomalies* with similar semantics: lost update, dirty write, and write skew.

A *lost update* occurs when transactions $T_1$ and $T_2$ both attempt to update the value of $V$. $T_1$ and $T_2$ read the value of $V$. $T_1$ updates $V$ and commts, and $T_2$ updates $V$ after that and commits as well. Since the transactions are not aware about each other's existence, if both of them are allowed to commit, the results of $T_1$ will be overwritten by the results of $T_2$, and the update from $T_1$ will be lost.

A *dirty write* is a situation in which one of the transactions takes an uncommitted value (i.e., dirty read), modifies it, and saves it. In other

words, when transaction results are based on the values that have never been committed.

A *write skew* occurs when each individual transaction respects the required invariants, but their combination does not satisfy these invariants. For example, transactions $T_1$ and $T_2$ modify values of two accounts $A_1$ and $A_2$. $A_1$ starts with `100$` and $A_2$ starts with `150$`. The account value is allowed to be negative, as long as the sum of the two accounts is nonnegative: $A_1 + A_2 >= 0$. $T_1$ and $T_2$ each attempt to withdraw `200$` from $A_1$ and $A_2$, respectively. Since at the time these transactions start $A_1 + A_2 = 250\$$, `250$` is available in total. Both transactions assume they're preserving the invariant and are allowed to commit. After the commit, $A_1$ has `-100$` and $A_2$ has `-50$`, which clearly violates the requirement to keep a sum of the accounts positive [FEKETE04].

## Isolation Levels

The lowest (in other words, weakest) isolation level is *read uncommitted*. Under this isolation level, the transactional system allows one transaction to observe uncommitted changes of other concurrent transactions. In other words, dirty reads are allowed.

We can avoid some of the anomalies. For example, we can make sure that any read performed by the specific transaction can only read *already committed* changes. However, it is not guaranteed that if the transaction attempts to read the same data record once again at a later stage, it will see the same value. If there was a committed modification between two reads, two queries in the same transaction would yield different results. In other words, dirty reads are not permitted, but phantom and nonrepeatable reads are. This isolation level is called *read committed*. If we further disallow nonrepeatable reads, we get a *repeatable read* isolation level.

The strongest isolation level is serializability. As we already discussed in "Serializability", it guarantees that transaction outcomes will appear in *some* order as if transactions were executed *serially* (i.e., without overlapping in time). Disallowing concurrent execution would have a substantial negative impact on the database performance. Transactions can get reordered, as long as their internal invariants hold and can be executed concurrently, but their outcomes have to appear in *some* serial order.

Figure 5-5 shows isolation levels and the anomalies they allow.

| | Dirty | Non-Repeatable | Phantom |
|---|---|---|---|
| Read Uncommitted | Allowed | Allowed | Allowed |
| Read Committed | - | Allowed | Allowed |
| Repeatable Read | - | - | Allowed |
| Serializable | - | - | - |

*Figure 5-5. Isolation levels and allowed anomalies*

Transactions that do not have dependencies can be executed in any order since their results are fully independent. Unlike linearizability (which we discuss in the context of distributed systems; see "Linearizability"), serializability is a property of *multiple* operations executed in *arbitrary*

order. It does not imply or attempt to impose any particular order on executing transactions. *Isolation* in ACID terms means serializability [BAILIS14a]. Unfortunately, implementing serializability requires coordination. In other words, transactions executing concurrently have to coordinate to preserve invariants and impose a serial order on conflicting executions [BAILIS14b].

Some databases use *snapshot isolation*. Under snapshot isolation, a transaction can observe the state changes performed by all transactions that were committed by the time it has started. Each transaction takes a snapshot of data and executes queries against it. This snapshot cannot change during transaction execution. The transaction commits only if the values it has modified did *not* change while it was executing. Otherwise, it is aborted and rolled back.

If two transactions attempt to modify the same value, only one of them is allowed to commit. This precludes a *lost update* anomaly. For example, transactions $T_1$ and $T_2$ both attempt to modify $V$. They read the current value of $V$ from the snapshot that contains changes from all transactions that were committed before they started. Whichever transaction attempts to commit first, will commit, and the other one will have to abort. The failed transactions will retry instead of overwriting the value.

A *write skew* anomaly is possible under snapshot isolation, since if two transactions read from local state, modify independent records, and preserve local invariants, they both are allowed to commit [FEKETE04]. We discuss snapshot isolation in more detail in the context of distributed transactions in “Distributed Transactions with Percolator”.

## Optimistic Concurrency Control

Optimistic concurrency control assumes that transaction conflicts occur rarely and, instead of using locks and blocking transaction execution, we can validate transactions to prevent read/write conflicts with concurrently executing transactions and ensure serializability before committing their results. Generally, transaction execution is split into three phases [WEIKUM01]:

*Read phase*

The transaction executes its steps in its own private context, without making any of the changes visible to other transactions. After this step, all transaction dependencies (*read set*) are known, as well as the side effects the transaction produces (*write set*).

*Validation phase*

Read and write sets of concurrent transactions are checked for the presence of possible conflicts between their operations that might violate serializability. If some of the data the transaction was reading is now out-of-date, or it would overwrite some of the values written by transactions that committed during its read phase, its private context is cleared and the read phase is restarted. In other words, the validation phase determines whether or not committing the transaction preserves ACID properties.

*Write phase*

If the validation phase hasn't determined any conflicts, the transaction can commit its write set from the private context to the database state.

Validation can be done by checking for conflicts with the transactions that have already been committed (*backward-oriented*), or with the transactions that are currently in the validation phase (*forward-oriented*). Validation and write phases of different transactions should be done atomically. No transaction is allowed to commit while some other transaction is being validated. Since validation and write phases are generally shorter than the read phase, this is an acceptable compromise.

Backward-oriented concurrency control ensures that for any pair of transactions $T_1$ and $T_2$, the following properties hold:

- $T_1$ was committed before the read phase of $T_2$ began, so $T_2$ is allowed to commit.
- $T_1$ was committed before the $T_2$ write phase, and the write set of $T_1$ doesn't intersect with the $T_2$ read set. In other words, $T_1$ hasn't written any values $T_2$ should have seen.
- The read phase of $T_1$ has completed before the read phase of $T_2$, and the write set of $T_2$ doesn't intersect with the read or write sets of $T_1$. In other words, transactions have operated on independent sets of data records, so both are allowed to commit.

This approach is efficient if validation usually succeeds and transactions don't have to be retried, since retries have a significant negative impact on performance. Of course, optimistic concurrency still has a *critical section*, which transactions can enter one at a time. Another approach that allows nonexclusive ownership for some operations is to use readers-writer locks (to allow shared access for readers) and upgradeable locks (to allow conversion of shared locks to exclusive when needed).

## Multiversion Concurrency Control

Multiversion concurrency control is a way to achieve transactional consistency in database management systems by allowing multiple record versions and using monotonically incremented transaction IDs or timestamps. This allows reads and writes to proceed with a minimal coordination on the storage level, since reads can continue accessing older values until the new ones are committed.

MVCC distinguishes between *committed* and *uncommitted* versions, which correspond to value versions of committed and uncommitted transactions. The last committed version of the value is assumed to be *current*. Generally, the goal of the transaction manager in this case is to have at most one uncommitted value at a time.

Depending on the isolation level implemented by the database system, read operations may or may not be allowed to access uncommitted values [WEIKUM01]. Multiversion concurrency can be implemented using locking, scheduling, and conflict resolution techniques (such as two-phase locking), or timestamp ordering. One of the major use cases for MVCC for implementing snapshot isolation [HELLERSTEIN07].

## Pessimistic Concurrency Control

Pessimistic concurrency control schemes are more conservative than optimistic ones. These schemes determine transaction conflicts while they're running and block or abort their execution.

One of the simplest pessimistic (lock-free) concurrency control schemes is *timestamp ordering*, where each transaction has a timestamp. Whether or not transaction operations are allowed to be executed is determined by whether or not any transaction with an *earlier* timestamp has already been committed. To implement that, the transaction manager has to maintain `max_read_timestamp` and `max_write_timestamp` per value, describing read and write operations executed by concurrent transactions.

*Read* operations that attempt to read a value with a timestamp lower than `max_write_timestamp` cause the transaction they belong to be aborted, since there's already a newer value, and allowing this operation would violate the transaction order.

Similarly, *write* operations with a timestamp lower than `max_read_timestamp` would conflict with a more recent read. However, *write* operations with a timestamp lower than `max_write_timestamp` are allowed, since we can safely ignore the outdated written values. This conjecture is commonly called the *Thomas Write Rule* [THOMAS79]. As soon as read or write operations are performed, the corresponding maximum timestamp values are updated. Aborted transactions restart with a *new* timestamp, since otherwise they're guaranteed to be aborted again [RAMAKRISHNAN03].

## Lock-Based Concurrency Control

Lock-based concurrency control schemes are a form of pessimistic concurrency control that uses explicit locks on the database objects rather than resolving schedules, like protocols such as timestamp ordering do. Some of the downsides of using locks are contention and scalability issues [REN16].

One of the most widespread lock-based techniques is *two-phase locking* (2PL), which separates lock management into two phases:

- The *growing phase* (also called the *expanding phase*), during which all locks required by the transaction are acquired and no locks are released.
- The *shrinking phase*, during which all locks acquired during the growing phase are released.

A rule that follows from these two definitions is that a transaction cannot acquire any locks as soon as it has released at least one of them. It's important to note that 2PL does not preclude transactions from executing steps during either one of these phases; however, some 2PL variants (such as conservative 2PL) do impose these limitations.

> **WARNING**
>
> Despite similar names, two-phase locking is a concept that is entirely different from two-phase commit (see "Two-Phase Commit"). Two-phase commit is a protocol used for distributed multipartition transactions, while two-phase locking is a concurrency control mechanism often used to implement serializability.

### Deadlocks

In locking protocols, transactions attempt to acquire locks on the database objects and, in case a lock cannot be granted immediately, a transaction has to wait until the lock is released. A situation may occur when two transactions, while attempting to acquire locks they require in order to proceed with execution, end up waiting for each other to release the other locks they hold. This situation is called a *deadlock*.

Figure 5-6 shows an example of a deadlock: $T_1$ holds lock $L_1$ and waits for lock $L_2$ to be released, while $T_2$ holds lock $L_2$ and waits for $L_1$ to be released.

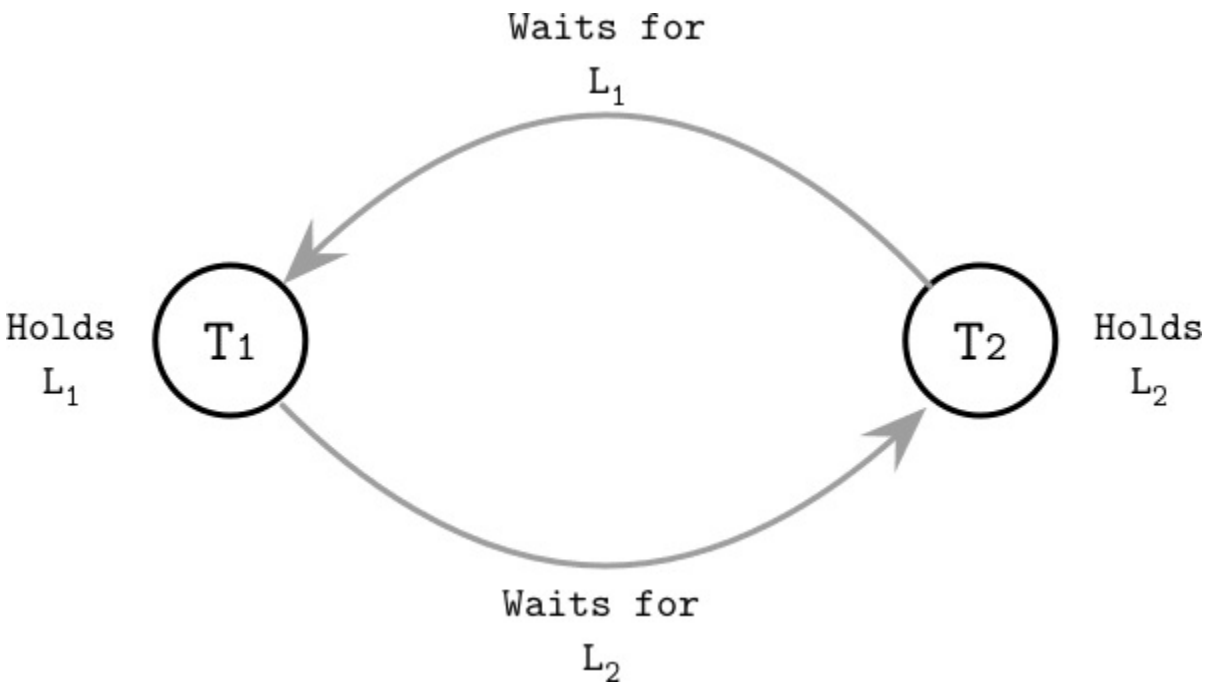


*Figure 5-6. Example of a deadlock*

The simplest way to handle deadlocks is to introduce timeouts and abort long-running transactions under the assumption that they might be in a deadlock. Another strategy, conservative 2PL, requires transactions to acquire all the locks before they can execute any of their operations and abort if they cannot. However, these approaches significantly limit system concurrency, and database systems mostly use a transaction manager to *detect* or *avoid* (in other words, prevent) deadlocks.

Detecting deadlocks is generally done using a *waits-for graph*, which tracks relationships between the in-flight transactions and establishes waits-for relationships between them.

Cycles in this graph indicate the presence of a deadlock: transaction $T_1$ is waiting for $T_2$ which, in turn, waits for $T_1$. Deadlock detection can be done *periodically* (once per time interval) or *continuously* (every time the waits-for graph is updated) [WEIKUM01]. One of the transactions (usually, the one that attempted to acquire the lock more recently) is aborted.

To *avoid* deadlocks and restrict lock acquisition to cases that will not result in a deadlock, the transaction manager can use transaction timestamps to determine their *priority*. A lower timestamp usually implies higher priority and vice versa.

If transaction $T_1$ attempts to acquire a lock currently held by $T_2$, and $T_1$ has higher priority (it started before $T_2$), we can use one of the following restrictions to avoid deadlocks [RAMAKRISHNAN03]:

*Wait-die*

> $T_1$ is allowed to block and *wait* for the lock. Otherwise, $T_1$ is aborted and restarted. In other words, a transaction can be blocked only by a transaction with a higher timestamp.

*Wound-wait*

> $T_2$ is aborted and restarted ($T_1$ *wounds* $T_2$). Otherwise (if $T_2$ has started before $T_1$), $T_1$ is allowed to wait. In other words, a transaction can be blocked only by a transaction with a lower timestamp.

Transaction processing requires a scheduler to handle deadlocks. At the same time, latches (see "Latches") rely on the programmer to ensure that deadlocks cannot happen and do not rely on deadlock avoidance mechanisms.

## Locks

If two transactions are submitted concurrently, modifying overlapping segments of data, neither one of them should observe partial results of the other one, hence maintaining logical consistency. Similarly, two threads from the same transaction have to observe the same database contents, and have access to each other's data.

In transaction processing, there's a distinction between the mechanisms that guard the logical and physical data integrity. The two concepts responsible logical and physical integrity are, correspondingly, *locks* and *latches*. The naming is somewhat unfortunate since what's called a latch here is usually referred to as a lock in systems programming, but we'll clarify the distinction and implications in this section.

Locks are used to isolate and schedule overlapping transactions and

manage database contents but not the internal storage structure, and are acquired on the key. Locks can guard either a specific key (whether it's existing or nonexisting) or a range of keys. Locks are generally stored and managed outside of the tree implementation and represent a higher-level concept, managed by the database lock manager.

Locks are more heavyweight than latches and are held for the duration of the transaction.

### Latches

On the other hand, latches guard the *physical* representation: leaf page contents are modified during insert, update, and delete operations. Nonleaf page contents and a tree structure are modified during operations resulting in splits and merges that propagate from leaf under- and overflows. Latches guard the physical tree representation (page contents and the tree structure) during these operations and are obtained on the page level. Any page has to be latched to allow safe concurrent access to it. Lockless concurrency control techniques still have to use latches.

Since a single modification on the leaf level might propagate to higher levels of the B-Tree, latches might have to be obtained on multiple levels. Executing queries should not be able to observe pages in an inconsistent state, such as incomplete writes or partial node splits, during which data might be present in both the source and target node, or not yet propagated to the parent.

The same rules apply to parent or sibling pointer updates. A general rule is to hold a latch for the smallest possible duration — namely, when the page is read or updated — to increase concurrency.

Interferences between concurrent operations can be roughly grouped into three categories:

- *Concurrent reads*, when several threads access the same page without modifying it.
- *Concurrent updates*, when several threads attempt to make modifications to the same page.
- *Reading while writing*, when one of the threads is trying to modify the page contents, and the other one is trying to access the same page

for a read.

These scenarios also apply to accesses that overlap with database maintenance (such as background processes, as described in “Vacuum and Maintenance”).

### Readers-writer lock

The simplest latch implementation would grant exclusive read/write access to the requesting thread. However, most of the time, we do not need to isolate *all* the processes from each other. For example, reads can access pages concurrently without causing any trouble, so we only need to make sure that multiple concurrent *writers* do not overlap, and *readers* do not overlap with *writers*. To achieve this level of granularity, we can use a *readers-writer lock* or RW lock.

An RW lock allows multiple readers to access the object concurrently, and only writers (which we usually have fewer of) have to obtain exclusive access to the object. Figure 5-7 shows the compatibility table for readers-writer locks: only readers can share lock ownership, while all other combinations of readers and writers should obtain exclusive ownership.

| | Reader | Writer |
|---|---|---|
| Reader | Shared | Exclusive |
| Writer | Exclusive | Exclusive |

*Figure 5-7. Readers-writer lock compatibility table*

In Figure 5-8 (a), we have multiple readers accessing the object, while the writer is waiting for its turn, since it can’t modify the page while readers access it. In Figure 5-8 (b), `writer 1` holds an exclusive lock on the object, while another writer and three readers have to wait.

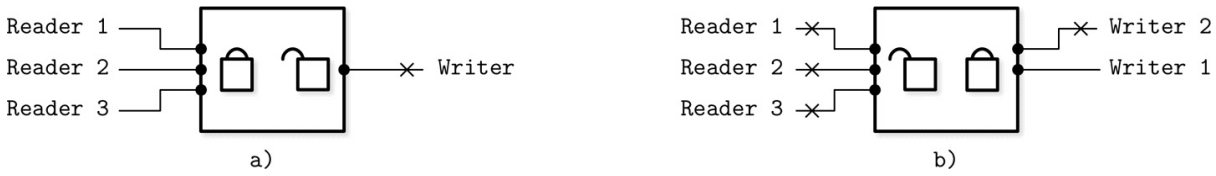


*Figure 5-8. Readers-writer locks*

Since two overlapping reads attempting to access the same page do not require synchronization other than preventing the page from being fetched from disk by the page cache twice, reads can be safely executed concurrently in shared mode. As soon as writes come into play, we need to isolate them from both concurrent reads and other writes.

**BUSY-WAIT AND QUEUEING TECHNIQUES**

To manage shared access to pages, we can either use blocking algorithms, which de-schedule threads and wake them up as soon as they can proceed, or use busy-wait algorithms. Busy-wait algorithms allow threads to wait for insignificant amounts of time instead of handing control back to the scheduler.

Queuing is usually implemented using compare-and-swap instructions, used to perform operations guaranteeing lock acquisition and queue update atomicity. If the queue is empty, the thread obtains access immediately. Otherwise, the thread appends itself to the waiting queue and spins on the variable that can be updated only by the thread preceding it in the queue. This helps to reduce the amount of CPU traffic for lock acquisition and release [MELLORCRUMMEY91].

## Latch crabbing

The most straightforward approach for latch acquisition is to grab all the latches on the way from the root to the target leaf. This creates a concurrency bottleneck and can be avoided in most cases. The time during which a latch is held should be minimized. One of the optimizations that can be used to achieve that is called *latch crabbing* (or latch coupling) [RAMAKRISHNAN03].

Latch crabbing is a rather simple method that allows holding latches for less time and releasing them as soon as it’s clear that the executing operation does not require them anymore. On the read path, as soon as the child node is located and its latch is acquired, the parent node’s latch can be released.

During insert, the parent latch can be released if the operation is

guaranteed not to result in structural changes that can propagate to it. In other words, the parent latch can be released if the child node is not full.

Similarly, during deletes, if the child node holds enough elements and the operation will not cause sibling nodes to merge, the latch on the parent node is released.

Figure 5-9 shows a root-to-leaf pass during insert:

- a) The write latch is acquired on the root level.
- b) The next-level node is located, and its write latch is acquired. The node is checked for potential structural changes. Since the node is not full, the parent latch can be released.
- c) The operation descends to the next level. The write latch is acquired, the target leaf node is checked for potential structural changes, and the parent latch is released.

This approach is optimistic: most insert and delete operations do not cause structural changes that propagate multiple levels up. In fact, the probability of structural changes decreases at higher levels. Most of the operations only require the latch on the target node, and the number of cases when the parent latch has to be retained is relatively small.

If the child page is still not loaded in the page cache, we can either latch a future loading page, or release a parent latch and restart the root-to-leaf pass after the page is loaded to reduce contention. Restarting root-to-leaf traversal sounds rather expensive, but in reality, we have to perform it rather infrequently, and can employ mechanisms to detect whether or not there were any structural changes at higher levels since the time of traversal [GRAEFE10].

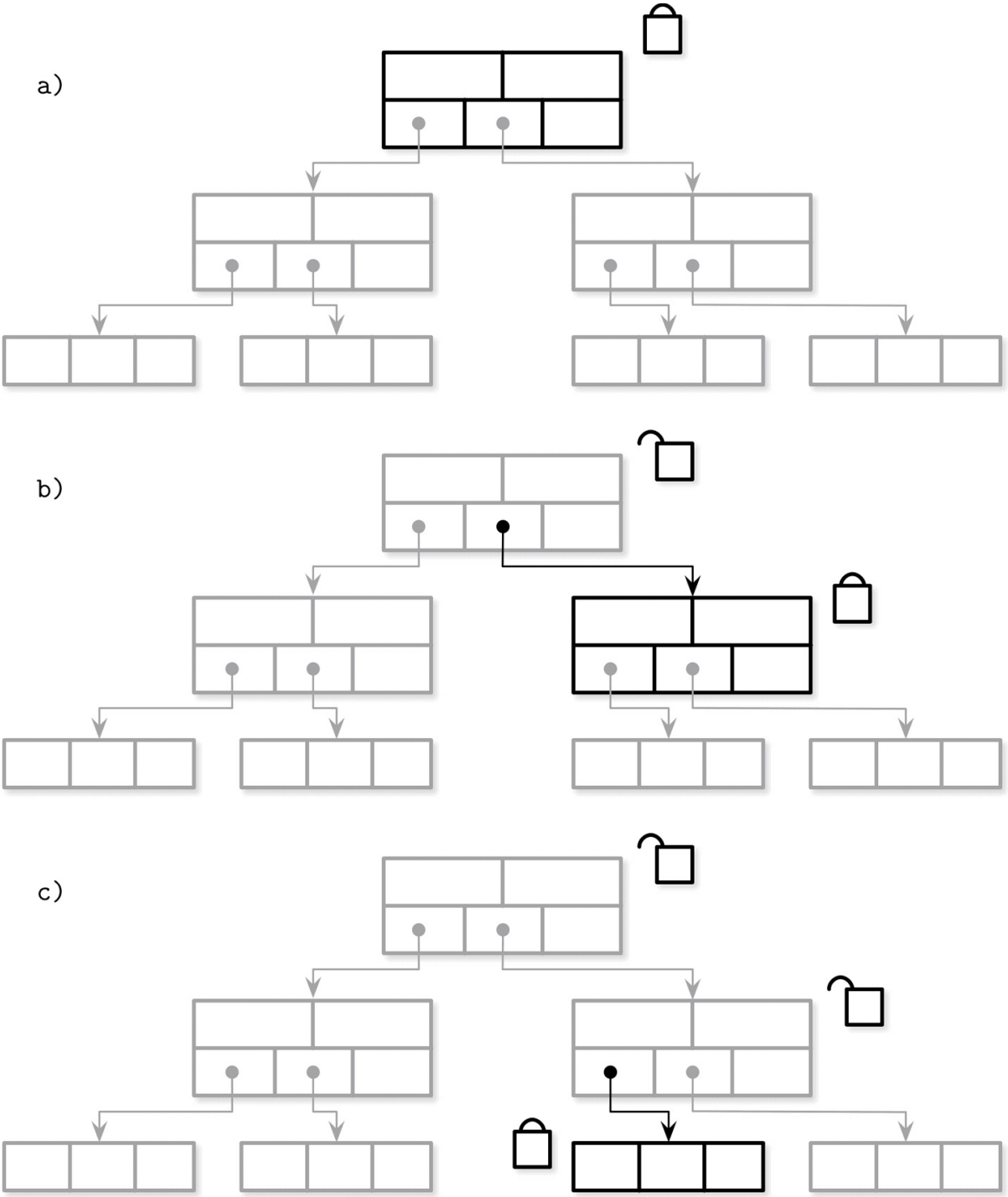


*Figure 5-9. Latch crabbing during insert*

### LATCH UPGRADING AND POINTER CHASING

Instead of acquiring latches during traversal in an exclusive mode right away, *latch upgrading* can be employed instead. This approach involves acquisition of shared locks along the search path and *upgrading* them to exclusive locks when necessary.

Write operations first acquire exclusive locks only at the leaf level. If the leaf has to be split or merged, the algorithm walks up the tree and attempts to *upgrade* a shared lock that the parent

holds, acquiring exclusive ownership of latches for the affected portion of the tree (i.e., nodes that will also be split or merged as a result of that operation). Since multiple threads might attempt to acquire exclusive locks on one of the higher levels, one of them has to wait or restart.

You might have noticed that the mechanisms described so far all start by acquiring a latch on the root node. Every request has to go through the root node, and it quickly becomes a bottleneck. At the same time, the root is always the last to be split, since all of its children have to fill up first. This means that the root node can *always* be latched optimistically, and the price of a retry (*pointer chasing*) is seldom paid.

## Blink-Trees

$B^{link}$-Trees build on top of B*-Trees (see "Rebalancing") and add *high keys* (see "Node High Keys") and *sibling link* pointers [LEHMAN81]. A high key indicates the highest possible subtree key. Every node but root in a $B^{link}$-Tree has two pointers: a child pointer descending from the parent and a sibling link from the left node residing on the same level.

$B^{link}$-Trees allow a state called *half-split*, where the node is already referenced by the sibling pointer, but not by the child pointer from its parent. Half-split is identified by checking the node high key. If the search key exceeds the high key of the node (which violates the high key invariant), the lookup algorithm concludes that the structure has been changed concurrently and follows the sibling link to proceed with the search.

The pointer has to be quickly added to the parent guarantee the best performance, but the search process doesn't have to be aborted and restarted, since all elements in the tree are accessible. The advantage here is that we do not have to hold the parent lock when descending to the child level, even if the child is going to be split: we can make a new node visible through its sibling link and update the parent pointer lazily without sacrificing correctness [GRAEFE10].

While this is slightly less efficient than descending directly from the parent and requires accessing an extra page, this results in correct root-to-leaf descent while simplifying concurrent access. Since splits are a relatively infrequent operation and B-Trees rarely shrink, this case is exceptional, and its cost is insignificant. This approach has quite a few benefits: it reduces contention, prevents holding a parent lock during splits, and reduces the number of locks held during tree structure modification to a

constant number. More importantly, it allows reads concurrent to structural tree changes, and prevents deadlocks otherwise resulting from concurrent modifications ascending to the parent nodes.

## Summary

In this chapter, we discussed the storage engine components responsible for transaction processing and recovery. When implementing transaction processing, we are presented with two problems:

- To improve efficiency, we need to allow concurrent transaction execution.
- To preserve correctness, we have to ensure that concurrently executing transactions preserve ACID properties.

Concurrent transaction execution can cause different kinds of read and write anomalies. Presence or absence of these anomalies is described and limited by implementing different isolation levels. Concurrency control approaches determine how transactions are scheduled and executed.

The page cache is responsible for reducing the number of disk accesses: it caches pages in memory and allows read and write access to them. When the cache reaches its capacity, pages are evicted and flushed back on disk. To make sure that unflushed changes are not lost in case of node crashes and to support transaction rollback, we use write-ahead logs. The page cache and write-ahead logs are coordinated using force and steal policies, ensuring that every transaction can be executed efficiently and rolled back without sacrificing durability.

**FURTHER READING**

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

*Transaction processing and recovery, generally*


Weikum, Gerhard, and Gottfried Vossen. 2001. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. San Francisco: Morgan Kaufmann Publishers Inc.

Bernstein, Philip A. and Eric Newcomer. 2009. *Principles of Transaction Processing*. San Francisco: Morgan Kaufmann.

Graefe, Goetz, Guy, Wey & Sauer, Caetano. 2016. "Instant Recovery with Write-Ahead Logging: Page Repair, System Restart, Media Restore, and System Failover, (2nd Ed.)" in *Synthesis Lectures on Data Management* 8, 1-113. 10.2200/S00710ED2V01Y201603DTM044.

Mohan, C., Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. "ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging." *Transactions on Database Systems* 17, no. 1

(March): 94-162. *https://doi.org/10.1145/128765.128770*.

*Concurrency control in B-Trees*

Wang, Paul. 1991. “An In-Depth Analysis of Concurrent B-Tree Algorithms.” MIT Technical Report. *https://apps.dtic.mil/dtic/tr/fulltext/u2/a232287.pdf*.
Goetz Graefe. 2010. A survey of B-tree locking techniques. ACM Trans. Database Syst. 35, 3, Article 16 (July 2010), 26 pages.

*Parallel and concurrent data structures*

McKenney, Paul E. 2012. “Is Parallel Programming Hard, And, If So,What Can You Do About It?” *https://arxiv.org/abs/1701.00854*.
Herlihy, Maurice and Nir Shavit. 2012. *The Art of Multiprocessor Programming, Revised Reprint (1st Ed.)*. San Francisco: Morgan Kaufmann.

*Chronological developments in the field of transaction processing*

Diaconu, Cristian, Craig Freedman, Erik Ismert, Per-Åke Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. “Hekaton: SQL Server’s Memory-Optimized OLTP Engine.” In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD ’13)*, 1243-1254. New York: Association for Computing Machinery. *https://doi.org/10.1145/2463676.2463710*.
Kimura, Hideaki. 2015. “FOEDUS: OLTP Engine for a Thousand Cores and NVRAM.” In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD ’15)*, 691-706. *https://doi.org/10.1145/2723372.2746480*.
Yu, Xiangyao, Andrew Pavlo, Daniel Sanchez, and Srinivas Devadas. 2016. “TicToc: Time Traveling Optimistic Concurrency Control.” In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD ’16)*, 1629-1642. *https://doi.org/10.1145/2882903.2882935*.
Kim, Kangnyeon, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. “ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads.” In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD ’16)*, 1675-1687. *https://doi.org/10.1145/2882903.2882905*.
Lim, Hyeontaek, Michael Kaminsky, and David G. Andersen. 2017. “Cicada: Dependably Fast Multi-Core In-Memory Transactions.” In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD ’17)*, 21-35. *https://doi.org/10.1145/3035918.3064015*.